

What Makes a Good Semantic ID for Generative Recommendation? A Reproducibility Study

Anonymous Author(s)

Abstract

Generative recommendation has recently emerged as a highly active research direction in recommender systems. A dominant practice in this paradigm is to represent items with *semantic IDs* (SIDs), which transform items into discrete codes that can be generated token by token. Although SIDs have shown promising empirical performance, their design remains highly diverse: existing studies adopt substantially different construction strategies, codebook organizations, and code lengths, often under different experimental settings. As a result, it remains unclear how semantic ID design truly affects generative recommendation performance.

In this work, we conduct a large-scale reproducibility study to systematically investigate the impact of semantic ID design on generative recommendation under a unified experimental framework. We focus on a fundamental question: *What makes a good semantic ID for generative recommendation?* To answer this question, we examine four aspects: the relative effectiveness of different semantic ID designs, the connection between codebook utilization and recommendation quality, the effect of semantic code length, and the influence of semantic ID design on item semantic learning. Through a unified evaluation on multiple benchmark datasets, we find that the effects of SID design are largely non-monotonic: no single SID design is universally best, and commonly used RQ-VAE and OPQ-based designs can behave inconsistently across datasets. Surprisingly, better codebook utilization does not necessarily lead to better recommendation performance, and scaling either the generative backbone or the SID length is not always beneficial. Semantic analysis further shows that OPQ and RQ-Kmeans appear to better preserve local item semantics, suggesting that effective SID design requires more than simply using more codes, longer identifiers, or larger models. Our study provides a controlled and reproducible understanding of semantic ID design, and offers practical insights for future generative recommender systems.¹

CCS Concepts

• Information systems → Recommender systems.

Keywords

Generative Recommendation, Semantic ID, Reproducibility

¹Our data and code are available at <https://anonymous.4open.science/t/recsys26-artifact-52FF>.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

ACM Reference Format:

Anonymous Author(s). 2018. What Makes a Good Semantic ID for Generative Recommendation? A Reproducibility Study. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 9 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Generative recommendation has become one of the most rapidly growing directions in recommender systems [10, 11, 20, 33]. Unlike traditional retrieve-then-rank pipelines, generative recommendation formulates recommendation as a sequence generation problem, where the target item is generated autoregressively as a sequence of tokens. This paradigm naturally leverages the strong generative capabilities of modern transformer models and provides an end-to-end solution for recommendation. A key technique that enables this paradigm is the use of *semantic IDs* (SIDs) [28], which replace arbitrary item identifiers with discrete codes derived from item semantics. Compared with randomly assigned IDs, SIDs carry richer semantic information and preserve meaningful relationships among items, making them more suitable for sequence generation. Moreover, by representing items with semantically grounded tokens, SIDs can better leverage the representational capacity and pre-trained knowledge of language models. As a consequence, SIDs have become a mainstream design choice in recent generative recommendation methods.

However, despite the popularity of semantic IDs, their design is still far from unified. Existing work has explored a broad range of semantic ID construction strategies, including various clustering-based, quantization-based, hierarchical, and learned discrete coding methods [28, 29, 32, 35]. While such diversity reflects the flexibility of semantic ID design, it also leads to a fundamental limitation in the current literature: there is still no systematic study that examines how different semantic ID designs affect generative recommendation under a unified experimental framework. Since prior studies are often conducted with different datasets, backbones, training recipes, and implementation details, the community still lacks a clear understanding of which design choices are truly effective and which observed improvements are confounded by inconsistent settings. To bridge this gap, we conduct a large-scale reproducibility study on semantic ID design for generative recommendation. We ask a key research question: *What makes a good semantic ID for generative recommendation?*

While the primary objective of semantic ID design is to improve recommendation performance, its effectiveness can be evaluated from multiple complementary aspects, such as codebook utilization, scaling behavior, and item semantic understanding. To comprehensively answer this core question, we further decompose it into the following four research questions:

RQ1. Which representative SID-based generative recommendation system performs better under a unified data split and

evaluation protocol? This question focuses on a method-level benchmark where all methods share the same preprocessing, data split, and evaluation scorer, while retaining their original training and inference designs.

RQ2. Which SID design has the best codebook utilization and does that lead to better recommendation performance? Codebook utilization is commonly viewed as an indicator of semantic ID quality, but its connection to downstream recommendation performance remains unclear. We therefore examine whether higher utilization consistently translates into better recommendation results.

RQ3. What are the scaling behaviors of generative recommendation based on different SIDs? We investigate the scaling properties of SID-based recommendation from two key perspectives: the SID length and the size of the generative recommender backbone. Specifically, we study how increasing SID length and enlarging the T5 model affect the representational capacity, generation difficulty, and overall recommendation performance.

RQ4. How do different semantic ID designs affect local item semantic preservation? Since semantic IDs are intended to encode meaningful item structure, it is important to assess whether different designs preserve local semantic neighborhoods in the induced SID space.

By answering these research questions, we aim to move the study of semantic IDs from fragmented empirical practice toward a more principled and reproducible understanding. To this end, we build a unified experimental framework and evaluate representative SID-based generative recommendation methods on multiple benchmark datasets. Our framework contains two levels of comparison: an overall method-level benchmark under a unified evaluation setup, and controlled SID-only analyses where the generative backbone and training protocol are fixed. Our results reveal that the effects of SID design are consistently non-monotonic. First, there is no universally best SID design: many methods are dataset-dependent, and even widely used RQ-VAE- and OPQ-based designs are not stable across all datasets. Second, surprisingly, better codebook utilization does not necessarily translate into better recommendation performance. Third, scaling is also non-monotonic: increasing either the generative backbone size or the SID length does not always improve results. Finally, from the semantic perspective, OPQ and RQ-Kmeans preserve local item semantics better than other compared designs.

Our contributions are summarized as follows:

- We present a large-scale reproducibility study on semantic ID design for generative recommendation, combining an overall method benchmark with controlled SID-only analyses under a unified preprocessing and evaluation protocol.
- We define a clear research agenda on semantic ID design and systematically examine four core questions: design effectiveness, codebook utilization, code scaling, and semantic learning.
- We provide practical insights into how to design effective semantic IDs for generative recommendation, offering empirical guidance for future research on generative recommender systems.

2 Related Work

Generative recommendation. Generative recommendation formulates recommendation as a conditional generation problem, where the target item is generated from user behavior sequences rather than selected only through a conventional retrieve-and-rank pipeline [5, 10, 15, 28]. Early text-to-text methods such as P5 unify different recommendation tasks with personalized prompts, while recent LLM-based recommenders further explore item generation, instruction tuning, and language-model adaptation for recommendation [2, 10, 15]. Although these methods demonstrate the flexibility of generation-based recommendation, they also reveal a fundamental target-representation challenge. Generating item titles or textual descriptions can introduce ambiguity, synonymy, and unnecessary language-generation difficulty, whereas generating atomic item IDs avoids textual ambiguity but provides little semantic sharing across related items. This motivates the use of structured item identifiers as generation targets. A representative method is TIGER, which introduces semantic IDs by mapping each item into a sequence of discrete codes and generating the target item identifier autoregressively [28]. Since then, semantic IDs have become a central interface between item representation and generative recommendation models.

Semantic ID design. Following TIGER, existing SID methods can be roughly organized by how they construct the discrete item code and how they incorporate collaborative information [1, 3, 7–9, 13, 18, 23, 26, 31, 34, 37]. From the construction perspective, TIGER uses RQ-VAE codes, while LETTER and ETEGRec further refine RQ-VAE-style tokenization with regularization or end-to-end learning [22, 28, 32]. Other methods explore different target spaces: RPG and DiffGRM adopt OPQ-based identifiers, OneRec uses RQ-Kmeans, HowToIndex relies on metadata/content-based indexing, EAGER builds hierarchical K-means behavior–semantic codes, SEATER constructs tree-structured SIDs, and SETRec uses an AE-based set tokenizer [6, 12, 14, 21, 24, 25, 29, 35]. This line shows a trend from simple autoregressive semantic codes toward more structured, parallel, hierarchical, and order-agnostic target spaces. From the preference-signal perspective, early SIDs mainly rely on content semantics, while later methods increasingly inject CF information: LETTER uses CF as tokenization regularization, SEATER incorporates CF into tree construction, SETRec and EAGER combine semantic and CF representations, and ETEGRec jointly learns the tokenizer and recommender with recommendation alignment [21, 22, 29, 32, 35]. Together, these studies show that SID design shapes not only the target space, but also semantic sharing, collaborative alignment, and decoding difficulty in generative recommendation.

The importance of SID design is further reflected in recent industrial generative recommendation systems. Large-scale systems such as OneRec show that generative recommendation is moving toward production deployment, while GLIDE and MBGR explicitly adopt semantic-ID-based or business-aware ID designs for large-scale retrieval and recommendation [4, 6, 16, 19]. However, existing SID methods are usually evaluated under different datasets, preprocessing pipelines, model backbones, and decoding implementations, making it difficult to isolate the effect of the identifier itself. Motivated by the growing importance of semantic IDs and the lack

Table 1: Statistics of the datasets used in our experiments.

Dataset	#Users	#Items	#Interactions
Video Games	74,333	25,062	666,397
MicroLens-50K	42,666	14,079	310,531
MicroLens-100K	90,171	17,228	658,056
Yelp	221,039	109,326	3,531,723

of systematic reproducible comparison, we treat SID design as the main object of study and evaluate representative SID designs under a unified framework.

3 Preliminaries

Task formulation. Let $\mathcal{U}$ and $\mathcal{I}$ denote the sets of users and items, respectively. For a user $u \in \mathcal{U}$, let $S_u^t = (i_1^u, \dots, i_t^u)$ denote the historical interaction sequence up to time t . The next-item recommendation task aims to predict the next item $i_{t+1}^u \in \mathcal{I}$ given S_u^t .

In semantic-ID-based generative recommendation, items are represented by sequences of discrete tokens instead of atomic item identifiers. A semantic-ID tokenizer $q : \mathcal{I} \rightarrow C_1 \times \dots \times C_L$ maps each item i to an L -token code sequence,

$$q(i) = (z_{i,1}, \dots, z_{i,L}), \quad z_{i,\ell} \in C_\ell.$$

Here C_ℓ denotes the codebook at level ℓ . The tokenizer q can be constructed using clustering, vector quantization, product quantization, tree-structured assignment, or learnable tokenization.

Given a user history S_u^t , the recommender first converts each interacted item into its semantic ID, yielding

$$Z_u^t = (q(i_1^u), \dots, q(i_t^u)),$$

where Z_u^t denotes the semantic ID sequence of user u up to time step t . The model then learns to generate the semantic ID of the target item. Assuming a one-to-one mapping between valid semantic IDs and items, the standard left-to-right autoregressive SID generation objective can be written as:

$$p_\theta(i_{t+1}^u | S_u^t) \equiv p_\theta(q(i_{t+1}^u) | Z_u^t) = \prod_{\ell=1}^L p_\theta(z_{i_{t+1},\ell}^u | Z_u^t, z_{i_{t+1},<\ell}^u),$$

where $z_{i_{t+1},<\ell}^u$ denotes the prefix tokens before level ℓ .

At inference time, the model generates valid semantic-ID sequences and maps them back to items through a code-to-item dictionary. Existing methods mainly differ in how the tokenizer q is designed and how the semantic ID is decoded. For example, some methods generate tokens from left to right, whereas others adopt constrained decoding, parallel prediction, or order-agnostic generation. The training objective is typically the negative log-likelihood of the target semantic ID, optionally combined with reconstruction, contrastive, diversity, or collaborative regularization losses.

Built upon this semantic-ID-based generative recommendation paradigm, this paper focuses on understanding how the design and properties of semantic IDs affect recommendation performance.

4 Experiment Setup

4.1 Dataset

We conduct experiments on multiple benchmark datasets, Amazon *Video Games*², *MicroLens*³, and *Yelp*⁴, covering three different recommendation scenarios and data characteristics. The detailed dataset statistics are presented in Table 1. The MicroLens dataset comprises two variants: we adopt MicroLens-50K as the default dataset, while MicroLens-100K is used for additional scaling evaluations. Following prior generative recommendation studies [21, 28, 30], we convert user-item interactions into chronological behavior sequences and adopt a unified global temporal split. Specifically, after applying the same filtering criteria to all methods, we merge all interactions into a global timestamp-ordered log and split it into training, validation, and test sets with a ratio of 70:13:17. This ensures that validation and test interactions occur after the training period at the global level, reducing future information leakage. After splitting, each user’s interactions are sorted chronologically to construct input sequences, with the next interacted item used as the prediction target. The same preprocessing pipeline is used for all baselines so that performance differences can be attributed to model and semantic ID design rather than data construction.

4.2 Evaluation

For fair comparison, we use the same global temporal split for all datasets: after 5-core filtering, interactions are sorted by timestamp and split by global time cutoffs, ensuring that test interactions occur after the training period [21]. Since each user may have multiple future interactions, we report multi-target Recall@5, Recall@10, NDCG@5, and NDCG@10, averaged uniformly across users. Our main protocol is warm-only: cold items are removed from the ground-truth set before evaluation, and users with no remaining ground-truth items are excluded. For Section 4.5 (RQ1), each method is evaluated under its original inference setting. For the controlled analyses from Section 5 (RQ2) to Section 7 (RQ4), we fix the TIGER framework and vary only the SID assignment. Specifically, we choose TIGER as the controlled backbone because it provides a simple and widely adopted autoregressive SID generation framework, allowing us to isolate the effect of the identifier assignment itself while keeping the training objective, decoding procedure, and evaluation pipeline unchanged. Following commonly adopted generative recommendation approaches [28, 32], we generate top- K predictions using constrained trie-based prefix decoding in these controlled experiments. The resulting candidates are passed through a shared post-processing pipeline that removes items appearing in the user’s training history and fills any remaining slots with training-time popular items, thereby guaranteeing K valid recommendations. All metrics are computed with the same strict scorer, so RQ1 reflects complete-method performance, while RQ2–RQ4 mainly reflect the effect of SID design.

4.3 Baselines

To provide a comprehensive and fair comparison, we include baselines from both **conventional sequential recommendation** and

²<https://amazon-reviews-2023.github.io>.

³<https://github.com/westlake-repl/MicroLens>.

⁴<https://business.yelp.com/data/resources/open-dataset/>

generative recommendation with semantic IDs. These methods cover major SID construction strategies, including RQ-VAE, OPQ, RQ-Kmeans, metadata/content-based indexing, hierarchical clustering, tree-based identifiers, AE-based set tokenization, and end-to-end learnable tokenization.

1. Conventional sequential recommendation.

- *SASRec* [17] is a strong self-attention-based sequential recommendation baseline.

2. Generative recommendation with semantic IDs.

- *TIGER* [28] is an RQ-VAE-based generative recommender that maps item semantic representations into residual-quantized code sequences and autoregressively generates target SIDs.
- *HowToIndex* [14] represents metadata/content-based item indexing. It constructs item identifiers from explicit item-side information or predefined indexing rules, providing a comparison to learned SID methods.
- *RPG* [12] is an OPQ-based method for long semantic IDs. It constructs unordered long SIDs through optimized product quantization and predicts SID tokens in parallel.
- *LETTER-TIGER* [28, 32] applies the LETTER tokenizer to the TIGER backbone. LETTER improves RQ-VAE tokenization with semantic regularization, CF-based contrastive regularization, and diversity regularization.
- *LETTER-LC-Rec* [32, 38] applies the same LETTER tokenizer to the LC-Rec backbone, allowing us to examine the effect of CF-regularized SID construction across different generative backbones.
- *SETER* [21] adopts an order-agnostic set identifier paradigm. It represents each item with CF and semantic tokens, where the semantic component is produced by an AE-based semantic tokenizer.
- *ETEGRec* [22] is an end-to-end learnable item tokenization framework that jointly optimizes an RQ-VAE-based tokenizer and a T5-like generative recommender with recommendation alignment objectives.
- *SEATER* [29] adopts a semantic tree-structured identifier design. It constructs balanced tree-based item identifiers with collaborative item embeddings and uses generation and contrastive learning objectives.
- *EAGER* [35] is a behavior-semantic two-stream generative recommender. It builds behavior and semantic codes through hierarchical K-means and generates them with two separate decoders.
- *OneRec* [6] represents a RQ-Kmeans-based SID construction strategy. It uses multi-level balanced quantization to generate semantic ID tokens and unifies retrieval and ranking in a generative framework.
- *DiffGRM* [25] is a diffusion-based generative recommendation model. It constructs SIDs with parallel semantic encoding and uses masked discrete diffusion with confidence-guided parallel denoising.

Overall, these baselines cover diverse SID construction strategies, decoding mechanisms, and CF incorporation designs, enabling a broad comparison of representative SID-based systems.

4.4 Implementation Details

For all compared methods, we mainly follow the hyperparameter settings, training recipes, and official implementations reported in their original papers to ensure a fair and reproducible comparison. For semantic-ID-based methods in Section 4.5, we keep the default tokenizer and codebook configurations from the corresponding original setups unless otherwise specified. From Section 5 (RQ2) to Section 7 (RQ4), we focus on semantic ID design. To enable meaningful comparisons and obtain clearer insights into SID behavior, we fix the SID implementation⁵ and the TIGER training protocol, varying only the SID design. Consequently, the performance of the same method may slightly differ between RQ1 and RQ2–RQ4. In the code-length scaling experiments, we vary the number of semantic ID levels as $L \in \{2, 3, 4, 6, 8, 12, 16\}$ for RQ-VAE-, RQ-Kmeans-, and OPQ-based identifiers. For backbone scaling, we instantiate the generative recommender using pre-trained T5-small, T5-base, and T5-large checkpoints [27]. All models are trained with early stopping based on validation NDCG@10, and the final results are reported on the test set using the same evaluation pipeline. Experiments are conducted on eight RTX 4090 GPUs.

4.5 Overall Benchmark (RQ1)

To answer RQ1 and investigate the complete-system performance of representative SID-based methods, we conduct a large-scale benchmark on public datasets: Video Games, Microlens, and Yelp. We first analyze the main results in Table 2, and then further discuss two aspects: SID construction strategies and CF signal incorporation.

The main observation from Table 2 is that no method dominates across all datasets. RPG is the strongest overall method, ranking first on Video Games and second on both Microlens and Yelp, but the best-performing method still varies by dataset: SEATER performs best on Microlens, while LETTER-TIGER performs best on Yelp. This dataset dependence is also visible for other methods, whose relative rankings change substantially across benchmarks. Meanwhile, SASRec remains a competitive lightweight baseline and outperforms several generative methods on Microlens and Yelp. This suggests that generative recommendation does not automatically outperform conventional sequential recommendation. The benefit depends critically on whether the SID design and the generative objective are well aligned with the data.

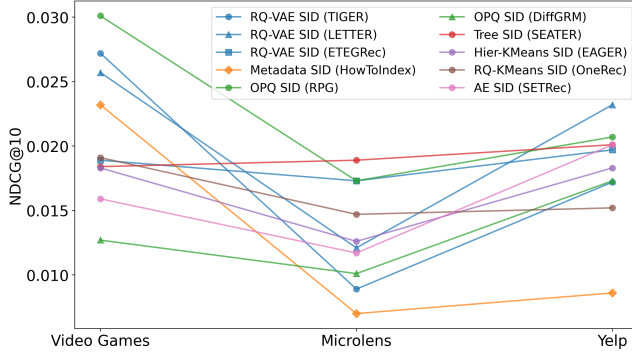
From the perspective of SID construction, Figure 1 shows that the construction family alone is insufficient to explain recommendation quality. Methods within the same family can behave quite differently. For example, OPQ-based methods show a large gap between RPG and DiffGRM, and RQ-VAE-based methods also exhibit clear dataset-dependent behavior. This indicates that SID effectiveness is not determined only by the high-level tokenizer type, such as RQ-VAE, OPQ, tree-based indexing, or metadata-based indexing. Instead, the full system design, including the tokenizer objective, code organization, decoding strategy, and its compatibility with the recommendation data, jointly determines performance.

From the CF signal exploitation perspective, Figure 2 shows that incorporating collaborative information is not uniformly beneficial. On Video Games, most CF-enhanced methods underperform

⁵<https://github.com/EdoardoBotta/RQ-VAE-Recommender>

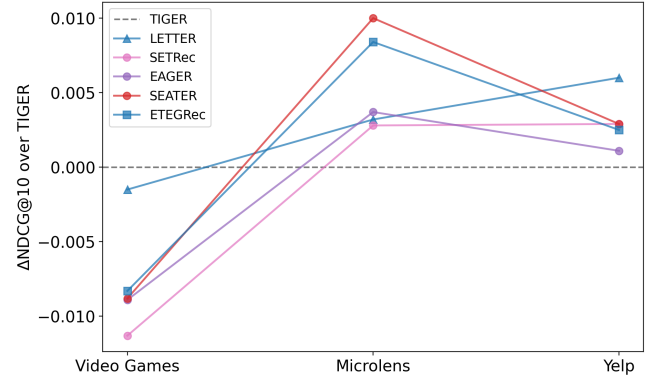
Table 2: Overall performance comparison under a unified data split and scorer, while preserving method-specific inference settings. The best-performing methods are shown in bold, and the second-best are underlined.

Model	Video Games				Microlens				Yelp			
	R@5	R@10	N@5	N@10	R@5	R@10	N@5	N@10	R@5	R@10	N@5	N@10
SASRec	0.0171	0.0300	0.0108	0.0152	0.0179	0.0318	0.0109	0.0156	0.0136	0.0257	0.0136	0.0177
TIGER	<u>0.0287</u>	<u>0.0468</u>	<u>0.0209</u>	<u>0.0272</u>	0.0099	0.0157	0.0069	0.0089	0.0138	0.0230	0.0144	0.0172
HowToIndex	0.0244	0.0369	0.0190	0.0232	0.0078	0.0129	0.0053	0.0070	0.0066	0.0110	0.0074	0.0086
RPG	0.0337	0.0477	0.0254	0.0301	<u>0.0193</u>	<u>0.0333</u>	<u>0.0126</u>	<u>0.0173</u>	<u>0.0169</u>	<u>0.0287</u>	<u>0.0169</u>	<u>0.0207</u>
LETTER-TIGER	0.0277	0.0437	0.0203	0.0257	0.0131	0.0231	0.0088	0.0121	0.0186	0.0322	0.0187	0.0232
LETTER-LC-Rec	0.0103	0.0187	0.0077	0.0105	0.0108	0.0176	0.0068	0.0091	0.0142	0.0232	0.0135	0.0167
SETRec	0.0167	0.0256	0.0128	0.0159	0.0127	0.0217	0.0087	0.0117	0.0157	0.0279	0.0162	0.0201
ETEGRec	0.0191	0.0320	0.0145	0.0189	0.0187	<u>0.0333</u>	0.0124	<u>0.0173</u>	0.0157	0.0275	0.0159	0.0197
SEATER	0.0206	0.0330	0.0142	0.0184	0.0221	0.0356	0.0143	0.0189	0.0154	0.0277	0.0162	0.0201
EAGER	0.0205	0.0311	0.0147	0.0183	0.0142	0.0229	0.0097	0.0126	0.0142	0.0252	0.0147	0.0183
DiffGRM	0.0149	0.0228	0.0101	0.0127	0.0117	0.0192	0.0076	0.0101	0.0134	0.0233	0.0142	0.0173
OneRec	0.0219	0.0322	0.0156	0.0191	0.0172	0.0265	0.0114	0.0147	0.0118	0.0194	0.0131	0.0152

**Figure 1: Dataset-wise NDCG@10 comparison of different SID-based methods on Video Games, Microlens, and Yelp. Each point denotes the result on one dataset, and lines connect the same SID design across datasets.**

the TIGER baseline, whereas on Microlens and Yelp several CF-enhanced methods bring clear gains. This suggests that CF signals can improve SID quality when collaborative patterns are reliable and aligned with item semantics, but may introduce noise or instability when such alignment is weak. Among the CF-incorporating methods, LETTER-TIGER shows relatively stable behavior, suggesting that using CF as a regularization signal during SID construction may be a more robust way to inject collaborative information than directly relying on CF representations throughout the full generative pipeline.

Answer to RQ1: The key conclusion is that SID design substantially affects generative recommendation performance, but the best design varies across datasets. This variation suggests that SID design should be adapted to different recommendation scenarios rather than treated as a universal choice. Different item semantics and user behavior patterns may require different identifier structures. First, RPG achieves the best overall benchmark performance, ranking first on Video Games and second on both Microlens and Yelp. Second, SASRec remains a strong baseline against generative methods on some datasets. Third, there is no universally

**Figure 2: Dataset-wise Δ NDCG@10 over TIGER for methods that incorporate collaborative filtering signals. Positive values indicate improvement over the baseline.**

best SID construction strategy; even widely used OPQ and RQ-VAE designs show large performance variation. Fourth, CF signal incorporation is also dataset-dependent. Among CF-incorporating methods, LETTER-TIGER shows the most stable behavior, suggesting that using CF information as a regularization signal during SID construction may be a relatively robust way to incorporate collaborative information.

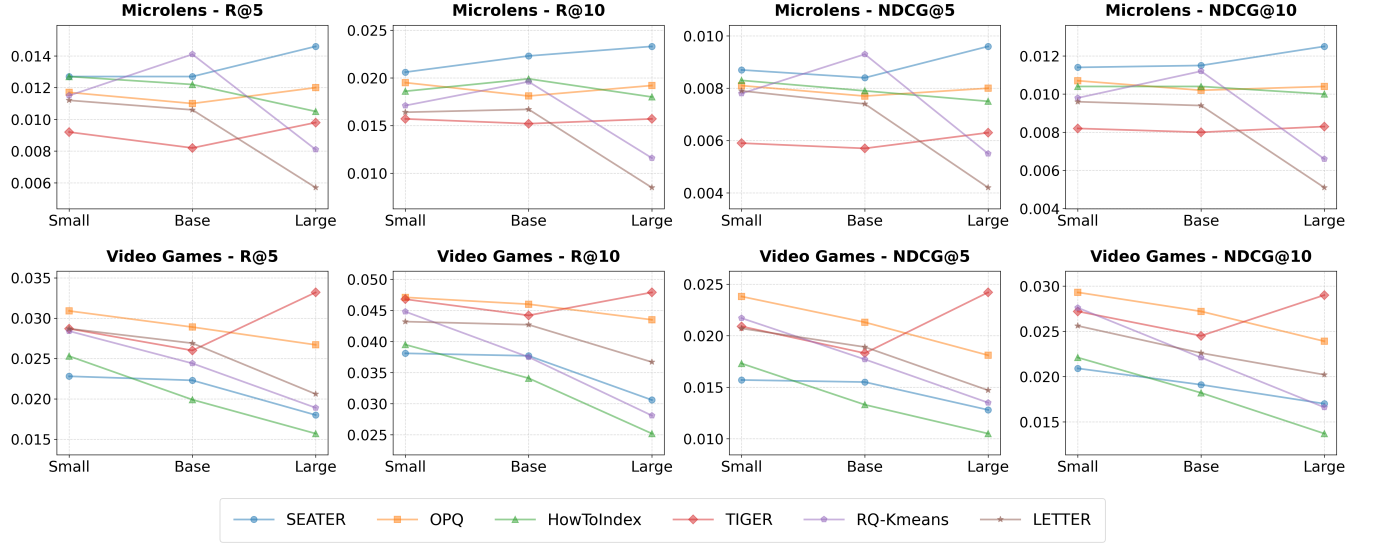
5 Codebook Utilization Investigation (RQ2)

To answer RQ2, we investigate whether better codebook utilization leads to better recommendation performance. Table 3 compares four representative identifier variants: TIGER (RQ-VAE), RQ-Kmeans, LETTER with diversity regularization, and LETTER without diversity regularization. All variants use a codebook with 256 possible codes for each of the three levels.

RQ-Kmeans achieves the highest L1 codebook utilization, while TIGER-RQ-VAE, LETTER-div, and LETTER-no-div use fewer first-level codes. However, the performance pattern is not monotonic with utilization: RQ-Kmeans does not consistently obtain the best ranking performance, and LETTER-no-div still achieves competitive results despite using the fewest L1 codes. The comparison

Table 3: Comparison of different SID design with the number of used codes at different codebook levels and performance on Video Games.

SID Design	L1 used	L2 used	L3 used	$R@5$	$R@10$	$N@5$	$N@10$
TIGER	161/256	252/256	256/256	<u>0.0287</u>	0.0468	0.0209	0.0272
RQ-Kmeans	256/256	256/256	256/256	0.0284	0.0448	0.0217	0.0276
LETTER-div	<u>170/256</u>	<u>254/256</u>	256/256	<u>0.0287</u>	0.0432	0.0207	0.0256
LETTER-no-div	98/256	248/256	256/256	0.0298	<u>0.0460</u>	<u>0.0212</u>	0.0276

**Figure 3: Scaling trends of different methods under small, base, and large T5 backbone on Video Games and Microlens. Each curve corresponds to one method, showing how its ranking performance changes as the model scale increases.****Table 4: Performance comparison under small, base, and large backbones of pre-trained T5 on Video Games and Microlens. The best and second-best results for each dataset, metric, and backbone are marked in bold and underlined, respectively.**

Dataset	SID Design	$R@5$			$R@10$			$N@5$			$N@10$		
		small	base	large	small	base	large	small	base	large	small	base	large
Microlens	TIGER	0.0092	0.0082	0.0098	0.0157	0.0152	0.0157	0.0059	0.0057	0.0063	0.0082	0.0080	0.0083
	SEATER	0.0127	<u>0.0127</u>	0.0146	0.0206	0.0223	0.0233	0.0087	<u>0.0084</u>	0.0096	0.0114	0.0115	0.0125
	OPQ	0.0117	0.0110	0.0120	0.0195	0.0181	0.0192	0.0081	0.0077	0.0080	0.0107	0.0102	0.0104
	HowToIndex	0.0127	0.0122	0.0105	0.0186	<u>0.0199</u>	0.0180	<u>0.0083</u>	0.0079	0.0075	0.0104	0.0104	0.0100
	LETTER	0.0112	0.0106	0.0057	0.0164	0.0167	0.0085	0.0079	0.0074	0.0042	0.0096	0.0094	0.0051
	RQ-Kmeans	0.0115	0.0141	0.0081	0.0171	0.0196	0.0116	0.0078	0.0093	0.0055	0.0098	<u>0.0112</u>	0.0066
Video Games	TIGER	0.0287	0.0260	0.0332	0.0468	0.0442	0.0479	0.0209	0.0183	0.0242	0.0272	0.0245	0.0290
	SEATER	0.0228	0.0223	0.0180	<u>0.0381</u>	0.0377	0.0306	0.0157	0.0155	0.0128	0.0209	0.0191	0.0170
	OPQ	0.0309	0.0289	<u>0.0267</u>	0.0471	0.0460	0.0435	0.0238	0.0213	<u>0.0181</u>	0.0293	0.0272	0.0239
	HowToIndex	0.0253	0.0199	<u>0.0157</u>	0.0395	0.0341	0.0252	0.0173	0.0133	0.0105	0.0221	0.0182	<u>0.0137</u>
	LETTER	0.0287	<u>0.0269</u>	0.0206	0.0432	0.0427	0.0367	0.0207	<u>0.0189</u>	0.0147	0.0256	0.0226	0.0202
	RQ-Kmeans	0.0284	0.0244	0.0189	0.0448	0.0375	0.0281	<u>0.0217</u>	0.0177	0.0135	<u>0.0276</u>	0.0221	0.0166

between LETTER-div and LETTER-no-div is especially informative, as diversity regularization substantially improves L1 utilization but does not improve downstream recommendation performance. Since the usage at deeper levels is very similar across methods, the main utilization difference lies in the first-level codebook. This suggests that L1 balancing is the most influential utilization factor, because it defines the coarse partition of the item space before finer-grained residual codes are assigned. These results show that codebook utilization, especially at L1, is a useful diagnostic signal, but it is not

sufficient to predict recommendation quality. Effective SID design should therefore balance L1 utilization with semantic coherence and generability, rather than maximizing code usage alone.

Answer to RQ2: Higher utilization does not consistently lead to better recommendation performance. RQ-Kmeans achieves the most balanced L1 utilization by using all 256 first-level codes, but its ranking performance is not consistently the best. Since the usage of deeper-level codebooks is already very similar across methods, the key utilization difference mainly comes from L1. Therefore,

Table 5: Performance comparison of different quantization methods under varying numbers of quantization levels L on Video Games. For each method, the best result in each metric is highlighted in bold, and the second-best result is underlined.

SID Design	L	$R@5$	$R@10$	$N@5$	$N@10$
RQ-VAE	2	0.0262	0.0448	0.0182	0.0255
RQ-VAE	3	0.0287	<u>0.0468</u>	0.0209	<u>0.0272</u>
RQ-VAE	4	0.0260	0.0409	<u>0.0203</u>	0.0255
RQ-VAE	6	0.0256	0.0440	<u>0.0187</u>	0.0248
RQ-VAE	8	<u>0.0270</u>	0.0458	0.0184	0.0257
RQ-VAE	12	<u>0.0266</u>	0.0511	0.0194	0.0277
RQ-VAE	16	0.0261	<u>0.0424</u>	0.0188	0.0244
RQ-Kmeans	2	0.0270	0.0443	0.0200	0.0252
RQ-Kmeans	3	0.0284	0.0448	0.0217	0.0276
RQ-Kmeans	4	0.0240	0.0369	0.0188	0.0233
RQ-Kmeans	6	0.0252	0.0364	0.0205	0.0241
RQ-Kmeans	8	0.0279	0.0416	<u>0.0208</u>	0.0255
RQ-Kmeans	12	<u>0.0214</u>	0.0297	<u>0.0179</u>	<u>0.0207</u>
RQ-Kmeans	16	0.0267	0.0400	0.0199	0.0244
OPQ	2	0.0292	0.0469	0.0208	0.0267
OPQ	3	0.0266	0.0413	0.0203	0.0253
OPQ	4	0.0320	0.0525	0.0230	0.0300
OPQ	6	0.0339	0.0556	0.0247	0.0321
OPQ	8	<u>0.0336</u>	0.0447	0.0246	0.0284
OPQ	12	0.0301	0.0433	0.0231	0.0275
OPQ	16	0.0242	0.0354	0.0188	0.0226

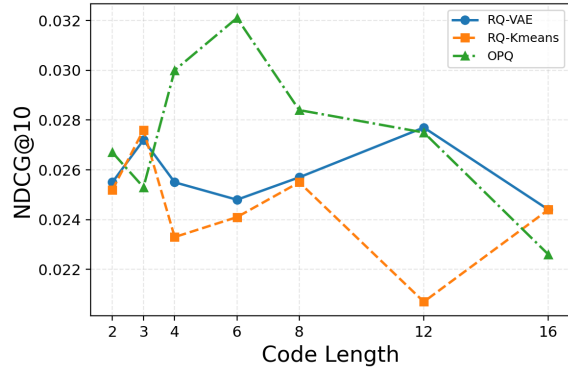


Figure 4: Scaling performance of different quantization methods with respect to code length on Video Games, evaluated by NDCG@10.

codebook utilization should be viewed as a useful but insufficient indicator of SID quality; effective semantic IDs should balance first-level utilization with semantic coherence and generability.

6 Scaling Analysis (RQ3)

To answer RQ3, we focus on controlled SID design scaling analysis. Among the compared SID designs, OPQ denotes the SID construction used by DiffGRM and RPG, while RQ-Kmeans denotes the SID construction used by OneRec. Other SID constructions have been discussed in RQ1 as complete method-level designs and are therefore referred to by their method names, such as SEATER, HowToIndex, LETTER, and TIGER. Under this controlled setting, we examine backbone scaling with T5-small/base/large and SID length scaling by varying the number of quantization levels L .

Table 6: Additional backbone scaling results on larger Microlens-100K. The best result for each method and metric is highlighted in bold.

SID Design	Backbone	$R@5$	$N@5$	$R@10$	$N@10$
LETTER	T5-small	0.0168	0.0113	0.0267	0.0148
	T5-base	0.0219	0.0137	0.0381	0.0191
	T5-large	0.0206	0.0134	0.0357	0.0186
RQ-Kmeans	T5-small	0.0177	0.0117	0.0262	0.0146
	T5-base	0.0215	0.0153	0.0352	0.0199
	T5-large	0.0208	0.0151	0.0330	0.0192

As shown in Figure 3 and Table 4, backbone scaling exhibits non-monotonic behavior across datasets and SID designs. Moving from T5-small to T5-large does not consistently improve performance. Some methods remain relatively stable, but others degrade substantially under larger backbones, especially on Video Games. This suggests that larger generative capacity alone is insufficient for SID-based recommendation, and may even amplify optimization instability or overfitting when the identifier design is not well matched to the data. Although our study only scales up to T5-large due to computational constraints, the results still reveal an important trend: increasing backbone capacity alone does not necessarily guarantee better SID-based recommendation performance.

To further examine whether this instability is related to data scale, we conduct additional experiments on Microlens-100K, as shown in Table 6. For the two examined SID variants, the degradation from T5-base to T5-large becomes milder on the larger dataset, and T5-base provides clear gains over T5-small. This indicates that data sparsity may partly contribute to unstable backbone scaling, although the trend remains dependent on the specific SID design. Therefore, the observed degradation should be interpreted as practical scaling instability under a controlled training recipe, rather than as evidence that larger backbones are inherently worse.

Table 5 and Figure 4 further show that SID length scaling is also non-monotonic. Different quantization methods prefer different identifier lengths: RQ-based designs generally favor compact codes, while OPQ benefits from moderately longer codes but degrades when the identifier becomes too long. This pattern reflects a trade-off between identifier capacity and generation difficulty. Longer SIDs can reduce collisions and provide finer item separation, but they also lengthen the autoregressive prediction path and enlarge the sparse valid-code space, making exact identifier generation harder.

Answer to RQ3: The key conclusion is that SID scaling is non-monotonic in both backbone size and code length. First, larger backbones do not consistently help; degradation is more evident on Video Games for several methods, while Microlens shows mixed behavior. Second, the additional Microlens-100K results show that this degradation can be partially alleviated with more data, suggesting that overfitting under limited data is one possible cause. Third, longer SIDs are not always better: RQ-Kmeans clearly favors compact codes around $L=3$, while RQ-VAE favors $L=3$ for top-5 metrics but reaches its best top-10 performance at $L=12$. OPQ performs best at a length $L=6$. Overall, effective SID scaling requires balancing backbone capacity, identifier capacity, and data scale rather than simply increasing model size or code length.

Table 7: Average top-20 semantic-neighborhood matching results across all items. Jaccard@20 measures strict top-20 set overlap, while RBO@20 measures rank-biased weighted top-20 agreement. Bold indicates the maximum values in each column, and underlining indicates the second-highest value.

SID Design	Jaccard@20	RBO@20
TIGER	0.1093	0.1416
SEATER	0.0637	0.0688
HowToIndex	0.0763	0.0915
LETTER	0.0885	0.0984
OPQ	0.1724	0.1806
RQ-Kmeans	<u>0.1280</u>	0.1967

7 Local Semantic Preservation Analysis (RQ4)

To answer RQ4, we examine whether different semantic ID designs preserve local item semantics in the induced SID space. Since semantic IDs are expected to encode meaningful item structure, a good SID should keep semantically related items close to each other. Unlike a purely qualitative case study, we evaluate local semantic preservation at both the anchor-level and the all-item level. Specifically, for each item, we retrieve its top-20 nearest neighbors in the reference semantic space and under each SID design, then compare the lists. We report Jaccard@20, the Jaccard similarity between the two top-20 sets, and RBO@20 [36], the Rank-Biased Overlap between the ranked lists.

Figure 5 provides an anchor-level visualization of local semantic neighborhoods, while Table 7 reports the all-item average results. The quantitative and qualitative results confirm that different SID designs vary substantially in their ability to preserve local item semantics. Among the compared designs, the OPQ-based SID achieves the highest Jaccard@20, indicating that it recovers the largest strict top-20 semantic-neighbor set overlap. In contrast, the RQ-Kmeans-based SID achieves the highest RBO@20, indicating stronger preservation of the rank order among top semantic neighbors.

This distinction suggests that OPQ and RQ-Kmeans preserve local semantics in different ways. The OPQ-based SID is stronger under strict set matching, meaning that it is better at retrieving the same semantic-neighborhood items regardless of their exact ranks. The RQ-Kmeans-based SID is stronger under rank-weighted matching, meaning that it better preserves highly ranked nearest neighbors near the top of the list. By contrast, SEATER obtains the lowest Jaccard@20 and RBO@20, suggesting that its tree-structured assignment may distort local semantic neighborhoods more strongly under this evaluation. TIGER, HowToIndex, and LETTER fall in the middle or lower range, showing that constructing valid discrete identifiers does not necessarily guarantee fine-grained neighborhood preservation.

Answer to RQ4: Across all items, OPQ-based and RQ-Kmeans-based SIDs provide the strongest local semantic preservation among the compared designs.

8 Conclusion and Future Work

This paper studies a central question in generative recommendation: what makes a good semantic ID? Through a unified reproducibility framework, we first benchmark representative SID-based methods

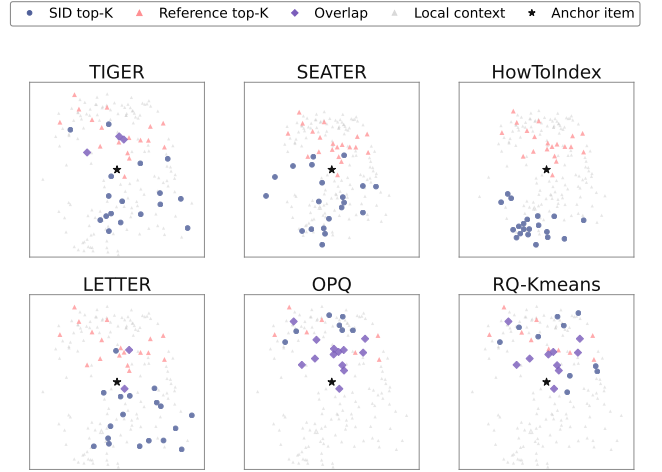


Figure 5: Visualization of local semantic neighborhoods for six SID designs on Video Games. The black star denotes the anchor item, gray triangles denote the local context, red triangles denote the reference top-K neighbors, blue circles denote the SID top-K neighbors, and purple diamonds denote the overlap between the reference and SID top-K sets.

under the same data split and evaluation scorer, and then conduct controlled SID-only analyses by fixing the generative framework. Our results show that SID design has substantial but largely non-monotonic effects. First, there is no universally best SID design: many methods are dataset-dependent, and common RQ-VAE and OPQ-based designs are not consistently stable across datasets. Second, higher codebook utilization does not necessarily improve recommendation performance, suggesting that utilization alone is insufficient for judging SID quality. Third, scaling is also non-monotonic: increasing either backbone size or SID length does not always help. Finally, semantically, OPQ and RQ-Kmeans better preserve local item semantics than other designs. Effective SID design should be judged by recommendation performance, scaling behavior, and preservation of item structure—not utilization, length, or model size alone.

These findings suggest several directions for future work. First, future SID methods should move beyond monotonic assumptions that higher codebook utilization, longer identifiers, or larger backbones are inherently better. Instead, SID tokenizers should be designed in an adaptive and dataset-aware manner, jointly considering data characteristics, identifier structure, and semantic preservation. Second, future empirical studies can build upon our unified framework by extending the scaling analysis to larger T5-based backbones, LLM-based generative recommenders, and broader recommendation scenarios, so as to examine whether the non-monotonic trends observed in this study persist under stronger generative capacity and more diverse data conditions. Finally, further analysis is needed to understand why popular SID families such as RQ-VAE and OPQ behave unstably across datasets, and to establish more robust evaluation protocols for reliable SID-based generative recommender systems.

References

- [1] Vladimir Baikalov, Iskander Bagautdinov, and Sergey Muravyov. 2026. Mitigating Collaborative Semantic ID Staleness in Generative Retrieval. *arXiv preprint arXiv:2604.13273* (2026).
- [2] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the 17th ACM conference on recommender systems*. 1007–1014.
- [3] Daiwei Chen, Zhoutong Fu, Guoyao Li, Rui Cai, Tan Wang, Yihan Cao, Ruijie Jiang, Haichao Zhang, Chengming Jiang, and Ramya Korlakai Vinayak. [n. d.]. STAR: Semantic-ID Token-Embedding Alignment For Generative Recommenders. ([n. d.]).
- [4] Edoardo D'Amico, Marco De Nadai, Praveen Chandar, Divita Vohra, Shawn Lin, Max Lefarov, Paul Gigioli, Gustavo Penha, Ilya Kopsysitsky, Ivo Joel Senese, et al. 2026. Deploying Semantic ID-based Generative Retrieval for Large-Scale Podcast Discovery at Spotify. *arXiv preprint arXiv:2603.17540* (2026).
- [5] Yashar Deldjoo, Zhankui He, Julian McAuley, Anton Korikov, Scott Sanner, Arnau Ramisa, Rene Vidal, Maheswaran Sathiamoorthy, Atoosa Kasrizadeh, Silvia Milano, et al. 2024. Recommendation with generative models. *arXiv preprint arXiv:2409.15173* (2024).
- [6] Jiaxin Deng, Shiyao Wang, Kuo Cai, Lejian Ren, Qigen Hu, Weifeng Ding, Qiang Luo, and Guorui Zhou. 2025. Onerec: Unifying retrieve and rank with generative recommender and iterative preference alignment. *arXiv preprint arXiv:2502.18965* (2025).
- [7] Dengzhao Fang, Jingtong Gao, Chengcheng Zhu, Yu Li, Xiangyu Zhao, and Yi Chang. 2025. Hid-vae: Interpretable generative recommendation via hierarchical and disentangled semantic ids. *arXiv preprint arXiv:2508.04618* (2025).
- [8] Junchen Fu, Xuri Ge, Alexandros Karatzoglou, Ioannis Arapakis, Suzan Verberne, Joemon M Jose, and Zhaochun Ren. 2026. Differentiable Semantic ID for Generative Recommendation. *arXiv preprint arXiv:2601.19711* (2026).
- [9] Kairui Fu, Tao Zhang, Shuwen Xiao, Ziyang Wang, Xinming Zhang, Chenchi Zhang, Yuliang Yan, Junjun Zheng, Yu Li, Zhihong Chen, et al. 2025. Forge: Forming semantic identifiers for generative retrieval in industrial datasets. *arXiv preprint arXiv:2509.20904* (2025).
- [10] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *Proceedings of the 16th ACM conference on recommender systems*. 299–315.
- [11] Min Hou, Le Wu, Yuxin Liao, Yonghui Yang, Zhen Zhang, Changlong Zheng, Han Wu, and Richang Hong. 2025. A survey on generative recommendation: Data, model, and tasks. *arXiv preprint arXiv:2510.27157* (2025).
- [12] Yupeng Hou, Jiacheng Li, Ashley Shin, Jinsung Jeon, Abhishek Santhanam, Wei Shao, Kaveh Hassani, Ning Yao, and Julian McAuley. 2025. Generating long semantic ids in parallel for recommendation. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 956–966.
- [13] Yupeng Hou, An Zhang, Leheng Sheng, Jiancan Wu, Xiang Wang, Tat-Seng Chua, and Julian McAuley. 2025. Towards large generative recommendation: A tokenization perspective. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 6821–6824.
- [14] Wenyue Hua, Shuyuan Xu, Yingqiang Ge, and Yongfeng Zhang. 2023. How to index item ids for recommendation foundation models. In *Proceedings of the Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*. 195–204.
- [15] Jianchao Ji, Zelong Li, Shuyuan Xu, Wenyue Hua, Yingqiang Ge, Juntao Tan, and Yongfeng Zhang. 2024. Genrec: Large language model for generative recommendation. In *European Conference on Information Retrieval*. Springer, 494–502.
- [16] Clark Mingxuan Ju, Tong Zhao, Leonardo Neves, Liam Collins, Bhuvish Kumar, Jiwen Ren, Lili Zhang, Wenfeng Zhuo, Vincent Zhang, Xiao Bai, et al. 2026. Semantic IDs for Recommender Systems at Snapchat: Use Cases, Technical Challenges, and Design Choices. *arXiv preprint arXiv:2604.03949* (2026).
- [17] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.
- [18] Kirill Khrylchenko. 2026. Variable-Length Semantic IDs for Recommender Systems. *arXiv preprint arXiv:2602.16375* (2026).
- [19] Changhao Li, Junwei Yin, Zhilin Zeng, Senjie Kou, Shuli Wang, Wenshuai Chen, Yinhua Zhu, Haitao Wang, and Xingxing Wang. 2026. MBGR: Multi-Business Prediction for Generative Recommendation at Meituan. *arXiv preprint arXiv:2604.02684* (2026).
- [20] Lei Li, Yongfeng Zhang, Dugang Liu, and Li Chen. 2024. Large language models for generative recommendation: A survey and visionary discussions. In *Proceedings of the 2024 joint international conference on computational linguistics, language resources and evaluation (LREC-COLING 2024)*. 10146–10159.
- [21] Xinyu Lin, Haihan Shi, Wenjie Wang, Fuli Feng, Qifan Wang, See-Kiong Ng, and Tat-Seng Chua. 2025. Order-agnostic identifier for large language model-based generative recommendation. In *Proceedings of the 48th international ACM SIGIR conference on research and development in information retrieval*. 1923–1933.
- [22] Enze Liu, Bowen Zheng, Cheng Ling, Lantao Hu, Han Li, and Wayne Xin Zhao. 2025. Generative recommender with end-to-end learnable item tokenization. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 729–739.
- [23] Han Liu, Yinwei Wei, Xueming Song, Wei Li, Guan, Yuan-Fang Li, and Liqiang Nie. 2024. Mmgrec: Multimodal generative recommendation with transformer model. *arXiv preprint arXiv:2404.16555* (2024).
- [24] Zhanyu Liu, Shiyao Wang, Xingmei Wang, Rongzhou Zhang, Jiaxin Deng, Honghui Bao, Jinghao Zhang, Wuchao Li, Pengfei Zheng, Xiangyu Wu, et al. 2025. Onerec-think: In-text reasoning for generative recommendation. *arXiv preprint arXiv:2510.11639* (2025).
- [25] Zhao Liu, Yichen Zhu, Yiqing Yang, Xiao Lv, Guoping Tang, Rui Huang, Qiang Luo, Ruiming Tang, and Guorui Zhou. 2026. Diffgrm: Diffusion-based generative recommendation model. In *Proceedings of the ACM Web Conference 2026*. 5853–5864.
- [26] Gustavo Penha, Edoardo D'Amico, Marco De Nadai, Enrico Palumbo, Alexandre Tamborrino, Ali Vardasbi, Max Lefarov, Shawn Lin, Timothy Heath, Francesco Fabbri, et al. 2025. Semantic ids for joint generative search and recommendation. In *Proceedings of the Nineteenth ACM Conference on Recommender Systems*. 1296–1301.
- [27] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research* 21, 140 (2020), 1–67.
- [28] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. 2023. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems* 36 (2023), 10299–10315.
- [29] Zihua Si, Zhongxiang Sun, Jiale Chen, Guozhang Chen, Xiaoxue Zang, Kai Zheng, Yang Song, Xiao Zhang, Jun Xu, and Kun Gai. 2024. Generative retrieval with semantic tree-structured identifiers and contrastive learning. In *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*. 154–163.
- [30] Zhu Sun, Di Yu, Hui Fang, Jie Yang, Xinghua Qu, Jie Zhang, and Cong Geng. 2020. Are we evaluating rigorously? benchmarking recommendation for reproducible evaluation and fair comparison. In *Proceedings of the 14th ACM Conference on Recommender Systems*. 23–32.
- [31] Federica Valeau, Odysseas Boufalis, Polytimi Gkotsi, Joshua Rosenthal, and David Vos. 2026. Efficient Optimization of Hierarchical Identifiers for Generative Recommendation. In *European Conference on Information Retrieval*. Springer, 82–97.
- [32] Wenjie Wang, Honghui Bao, Xinyu Lin, Jizhi Zhang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Learnable item tokenization for generative recommendation. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 2400–2409.
- [33] Wenjie Wang, Xinyu Lin, Fuli Feng, Xiangnan He, and Tat-Seng Chua. 2023. Generative recommendation: Towards next-generation recommender paradigm. *arXiv preprint arXiv:2304.03516* (2023).
- [34] Yige Wang, Mingming Li, Li Wang, Kaichen Zhao, Wangming Li, Weipeng Jiang, and Xueying Li. 2026. MoMoREC: A Multi-agent Motivation Generation Framework for Residual Semantic ID-Aware Recommendation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 15904–15914.
- [35] Ye Wang, Jiahao Xun, Minjie Hong, Jieming Zhu, Tao Jin, Wang Lin, Haoyuan Li, Linjun Li, Yan Xia, Zhou Zhao, et al. 2024. Eager: Two-stream generative recommender with behavior-semantic collaboration. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3245–3254.
- [36] William Webber, Alistair Moffat, and Justin Zobel. 2010. A similarity measure for indefinite rankings. *ACM Transactions on Information Systems (TOIS)* 28, 4 (2010), 1–38.
- [37] Zhen Zhang, Jujia Zhao, Xinyu Ma, Xin Xin, Maarten de Rijke, and Zhaochun Ren. 2026. Cold-Starts in Generative Recommendation: A Reproducibility Study. *arXiv preprint arXiv:2603.29845* (2026).
- [38] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2024. Adapting large language models by integrating collaborative semantics for recommendation. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 1435–1448.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009